Machine Learning

Yinglong Zhang

April 17, 2019

Overview

1. Strong and week learning-Boosting
2. Stochastic gradient descent
3. Combining sleep expert advice
4. Further directions

Strong and week learning-Boosting

Aim: To boost a week learner to a strong learner by using the techniques of boosting.

Running the week learner on these different weights of the training sample will produce a series of $h_1, h_2, \cdots$, and the idea of reweighting procedure will be to focus attention on the parts of samples that previous hypotheses have performed poorly on. At the end, combing the hypotheses together by a majority vote.

- Strong learner: an algorithm that with high probability is able to achieve any desired error rate ϵ.

- Week learner: an algorithm that with high probability an error rate is less than or equal to $\frac{1}{2} - \gamma$ for some $0 < \gamma \leq \frac{1}{2}$.

Definition (γ-*week learner on sample*)

A γ-week learner is an algorithm that given examples. their labels, and a non-negative real weight w_i on each examples x_i, produces a classifier that correctly lables a subset of examples with total weight at least $(\frac{1}{2} + \gamma) \sum_{i=1}^{n} w_i$.

Boosting algorithm: Given a sample S of n labeled examples $x_1, \cdots, x_n$, initialize each example x_i to have weight $w_i = 1$. Let $\mathbf{w} = (w_1, \cdots, w_n)$.

For $t = 1, 2, \cdots, t_0$ do

Call the weak leaner on the weighted sample $(S, \mathbf{w})$, receiving hypothesis h_t. Multiply the weight of each example that was misclassified by h_t by $\alpha = \frac{\frac{1}{2}+\gamma}{\frac{1}{2}-\gamma}$. Leave the other weights as they are.

End

Output the classifier $MAJ(h_1, \cdots, h_{t_0})$ which takes the majority vote of the hypothesis returned by the week learner. Assume t_0 is odd so there is no tie.

Theorem 5.21

Let A be a γ-week learner for sample S. Then $t_0 = \mathcal{O}(\frac{1}{\gamma^2} \log n)$ is sufficient so that the classifier $MAJ(h_1, \cdots, h_{t_0})$ produced by the boosting procedure has training error zero.

Proof

• On one hand, suppose the number of examples the final classifier gets wrong is m.

Each of these m examples was mis-classified at least $\frac{t_0}{2}$ times

$\Rightarrow$ each has weight at least $\alpha^{\frac{t_0}{2}}$ $\Rightarrow$ the total wight is at least $m\alpha^{\frac{t_0}{2}}$.

• One the other hand, let $weight(t)$ be the total weight at time t.

The total weight of mis-classified examples of the weight at time t is at most $(\frac{1}{2} - \gamma)$

$$\Rightarrow \quad \mathit{weight}(t+1) \leq \left(\alpha(\frac{1}{2}-\gamma)+(\frac{1}{2}+)\right) \times \mathit{weight}(t)$$
$$= (1+2\gamma) \times \mathit{weight}(t).$$

Together with $\mathit{weight}(0) = n \quad \Rightarrow \quad$ the total weight is at most $n(1+2\gamma)^{t_0}$.

$$\Rightarrow \quad m\alpha^{\frac{t_0}{2}} \leq \mathit{total\ weight\ at\ end} \leq n(1+2\gamma)^{t_0}.$$

Substituting $\alpha = \frac{\frac{1}{2}+\gamma}{\frac{1}{2}-\gamma} = \frac{1+2\gamma}{1-2\gamma}$, we have

$$m \leq n(1-2\gamma)^{\frac{t_0}{2}}(1+2\gamma)^{\frac{t_0}{2}} = (1-4\gamma^2)^{\frac{t_0}{2}}.$$

By using relation $1-x \leq e^{-x} \quad \Rightarrow \quad m \leq ne^{-2t_0\gamma^2}$. Hence

$$m \leq ne^{-2t_0\gamma^2} \leq 1 \quad \Rightarrow \quad t_0 > \frac{\ln n}{2\gamma^2}. \qquad \square$$

Stochastic gradient descent

The widely used algorithm in machine learning is Stochastic gradient descent.

• Let $\mathcal{F}$ be a class of real-valued function $f_{\mathbf{w}} : \mathbb{R}^d \to \mathbb{R}$ where $\mathbf{w} = (w_1, \cdots, w_n)$ is a vector of parameter. e.g., if $n = d$, we can take $f_{\mathbf{w}}(x) = \mathbf{w}^T \mathbf{x}$.

• Let $L(f_{\mathbf{w}}(\mathbf{x}), c^*(\mathbf{x}))$ be a loss function describing the real-valued penalty associating with function $f_{\mathbf{w}}(\mathbf{x})$ for its prediction on an example $\mathbf{x}$ whose true label is $c^*(\mathbf{x})$.

Stochastic Gradient Descent:

Given: starting point $\mathbf{w} = \mathbf{w}_{init}$ and leaning rates $\lambda_1, \lambda_2, \cdots$,

Consider a sequence of random examples $(\mathbf{x}_1, c^*(\mathbf{x}_1))$, $(\mathbf{x}_1, c^*(\mathbf{x}_1)), \cdots$

1. Given example $(\mathbf{x}_t, c^*(\mathbf{x}_t))$, compute the gradient $\nabla L(f_{\mathbf{w}}(\mathbf{x}_t), c^*(\mathbf{x}_t)) = \left(\frac{\partial L(f_{\mathbf{w}}(\mathbf{x}_t), c^*(\mathbf{x}_t))}{\partial w_1}, \cdots, \frac{\partial L(f_{\mathbf{w}}(\mathbf{x}_t), c^*(\mathbf{x}_t))}{\partial w_n}\right)$.

2. Update: $\mathbf{w} \leftarrow \mathbf{w} - \lambda_t \nabla L(f_{\mathbf{w}}(\mathbf{x}_t), c^*(\mathbf{x}_t))$.

Combining sleep expert advice

- Sleeping expert: a predictor h that on any given example $\mathbf{x}$ either makes a prediction on its label or chooses to stay silent.
- $mistake(A, S)$: the number of mistakes of an algorithm A on a sequence of examples S.
- The case of predictors that make prediction on every example is called the problem of combing expert advice.
- The case of f predictors that sometimes fire and sometimes are silent is called the sleeping expert problem.

Suppose we have access to n sleeping experts $h_1, \cdots, h_n$ from a concept class $\mathcal{H}$.

Let S_i denote the subset of examples on which h_i makes a prediction.

Combing sleeping experts algorithm:

Initialize each expert h_i with a weight $w_i = 1$. Let $\varepsilon \in (0, 1)$. For each example x, do the following:

1. [Make prediction] Let H_x denote the set of experts h_i that make a prediction on x. Let $w_x = \sum_{h_j \in H_x} w_j$. Choose $h_i \in H_x$ with probability $p_{ix} = \frac{w_i}{w_x}$ and predict $h_i(x)$.

2. [Receive feedback] For each $h_i \in H_x$, let $m_{ix} = 1$ if $h_i(x)$ is incorrect, else let $m_{ix} = 0$.

3. [Update weights] For each $h_i \in H_x$, update its weight as follows

- *Let* $r_{ix} = \frac{\sum_{h_j \in H_x} p_{jx} m_{jx}}{1+\varepsilon} - m_{ix}$,
  where $\sum_{h_j \in H_x} p_{jx} m_{jx}$ *represents the probability of algorithm making a mistake on example* x, *i.e.,* $\mathbb{E}\big(mistake(A, x)\big)$.

- *Update* $w_i \quad \leftarrow \quad w_i(1+\varepsilon)^{r_{ix}}$.

For each $x_i \in H_x$, leave w_i alone.

Theorem 5.22

For any set of n sleeping experts $h_1, \cdots, h_n$, and for any sequence of examples S, the combining sleeping experts algorithm A satisfies for all i:

$$\mathbb{E}\big(mistake(A, S_i)\big) \leq (1+\varepsilon) \cdot mistake(h_i, S_i) + \mathcal{O}(\frac{\log n}{\varepsilon}).$$

Proof

Consider the sleeping expert h_i. The weight of h_i after the sequence of examples S is

$$w_i = (1+\varepsilon)^{\sum_{x \in S_i} \left\{ \frac{\sum_{h_j \in H_x} p_{jx} m_{jx}}{1+\varepsilon} - m_{ix} \right\}}$$
$$= (1+\varepsilon)^{\frac{\mathbb{E}\left(mistake(A,S_i)\right)}{1+\varepsilon} - mistake(h_i,S_i)}.$$

By taking logs, we have

$$\frac{\mathbb{E}\big(mistake(A,S_i)\big)}{1+\varepsilon} - mistake(h_i,S_i) = \frac{\log w_i}{\log(1+\varepsilon)}.$$

Now let $w = \sum_j w_j$. then $w_i \leq w$. Hence, we have

Now let $w = \sum_j w_j$. then $w_i \leq w$. Hence, we have

$$\frac{\mathbb{E}\big(mistake(A, S_i)\big)}{1+\varepsilon} - mistake(h_i, S_i) \leq \frac{\log w}{\log(1+\varepsilon)} = \mathcal{O}(\frac{\log w}{\varepsilon}).$$

Thus, we have

$$\mathbb{E}\big(mistake(A, S_i)\big) \leq (1+\varepsilon) \cdot mistake(h_i, S_i) + \mathcal{O}(\frac{\log w}{\varepsilon}).$$

Since $w = n$ initially. Hence, we only need to prove w is not increasing.

It is sufficient to prove

$$\sum_{h_i \in H_x} w_i (1+\varepsilon)^{r_{ix}} \leq \sum_{h_i \in H_x} w_i \quad \Rightarrow \quad \sum_i p_{ix} (1+\varepsilon)^{r_{ix}} \leq 1,$$

where $p_{ix} = 0$ for $h_i \notin H_x$. Recall that

$$\sum_i p_{ix} (1+\varepsilon)^{r_{ix}} = \sum_i p_{ix} (1+\varepsilon)^{\frac{\sum_j p_{jx} m_{jx}}{1+\varepsilon} - m_{ix}}.$$

Set $\beta = \frac{1}{1+\varepsilon}$, then

$$\sum_i p_{ix} (1+\varepsilon)^{r_{ix}} = \sum_i p_{ix} \beta^{m_{ix} - \left(\sum_j p_{jx} m_{jx}\right)\beta}.$$

By using relations:

$$\beta^z \leq 1 - (1-\beta)z, \quad \beta^{-z} \leq 1 + \frac{(1-\beta)z}{\beta}, \quad \beta, z \in [0,1].$$

we have

$$\begin{aligned}\sum_i p_{ix}(1+\varepsilon)^{r_{ix}} &\leq \sum_i p_{ix}\Big(1-(1-\beta)m_{ix}\Big)\big(1+(1-\beta)(\sum_j p_{jx}m_{jx})\big)\\ &\leq (\sum_i p_{ix}) - (1-\beta)\sum_i p_{ix}m_{ix}\\ &+ (1-\beta)\sum_i p_{ix}\sum_j p_{jx}m_{jx}\\ &= 1.\end{aligned}$$

Semi-supervised learning

• Idea: to use a large unlabeled data set U to argue a given labeled data set L in order to produce more accurate rules than would have been achieved just using L alone.

- Notation of compatibility between a hypothesis $h \in \mathcal{H}$ and a distribution $\mathcal{D} \in \mathcal{X}$:

  - $\chi(h, \mathcal{D}) : \mathcal{H} \times \mathcal{X} \rightarrow [0, 1]$.
  - $\chi(h, \mathcal{D}) = 1$ means that h is highly compatible with $\mathcal{D}$.
  - $\chi(h, \mathcal{D}) = 0$ means that h is very incompatible with $\mathcal{D}$.

- $err_{unl}(h) = 1 - \chi(h, \mathcal{D})$ is called the unlabeled error rate of h. For convenience, we also overload notation χ as an expectation over individual examples: $\chi(h, \mathcal{D}) = \mathbb{E}_{x \sim \mathcal{D}}[\chi(h, x)]$.

- $err_{unl}(c^*) = 0$ means the target is fully compatible.

- The case that $c^* \in \mathcal{H}$ and $err_{unl}(c^*) = 0$ is termed the "doubly realizable case".

Now fix concept functions $\mathcal{H}$ and compatibility notation χ. For given labeled sample L and unlabeled sample U:

- $\widehat{err}(h)$ is the fraction of mistake of h on L, i.e., the empirical error rate of h.
- $\widehat{err}_{unl}(h) = 1 - \chi(h, U)$ with $\chi(h, U) = \mathbb{E}_{x \sim U}[\chi(h, x)]$ is the unlabeled error rate of h.
- For given $\alpha > 0$, define $\mathcal{H}_{\mathcal{D},\chi}(\alpha)$ to be the set of functions $f \in \mathcal{H}$ such that $err_{unl}(f) \leq \alpha$.

Theorem 5.23

If $c^* \in \mathcal{H}$ then with probability at least $1-\delta$, for labeled set L and unlabeled set U drawn form $\mathcal{D}$, the $h \in \mathcal{H}$ that optimizes $\widehat{err}_{unl}(h)$ subject to $\widehat{err}(h) = 0$ will have $err_{\mathcal{D}} \leq \varepsilon$ for

$$|U| \geq \frac{2}{\varepsilon^2}\{\ln|\mathcal{H}| + \ln\frac{4}{\delta}\},$$
$$|L| \geq \frac{1}{\varepsilon}\{\ln|\mathcal{H}_{\mathcal{D}}(err_{unl}(c^*)+2\varepsilon)| + \frac{2}{\delta}\}.$$

Equivalently, for $|U|$ satisfying this bound and for any $|L|$, the $h \in \mathcal{H}$ that minimizes $\widehat{err}_{unl}(h)$ subject to $\widehat{err}(h) = 0$ has

$$err_{\mathcal{D}}(h) \leq \frac{1}{|L|}\{\ln|\mathcal{H}_{\mathcal{D}}(err_{unl}(c^*)+2\varepsilon)| + \frac{2}{\delta}\}.$$

Others

- Active learning: Algorithms that take an active role in the selection of which examples are labeled. The aim is to reach a desired error rate ε using much fewer labels than would be needed by just labeling random examples.

- Multi-task learning: To learn multiple target functions $c_1^*, \cdots, c_n^*$.

Thanks for your attention!